



# Framework Profissional de Automação de Testes

---

# Construindo uma Arquitetura Enterprise do Zero

*Este material acompanha o projeto de referência. Cada parte corresponde a uma etapa da construção do framework, desde a fundação até a integração contínua.*

---

# Sumário

- Parte 1 — Visão Geral e Arquitetura
  - 1.1 O que é automação de testes e por que automatizar
  - 1.2 O que é um framework
  - 1.3 Pirâmide de testes
  - 1.4 Arquitetura em camadas
  - 1.5 Fluxo completo de execução
  - 1.6 Responsabilidade de cada camada
  - 1.7 Decisões tecnológicas e justificativas
  - 1.8 Estrutura completa de diretórios
  - 1.9 Pré-requisitos
- Parte 2 — Construção do Framework UI
  - 2.1 Criação do projeto Maven
  - 2.2 DriverFactory
  - 2.3 DriverManager
  - 2.4 BasePage
  - 2.5 LoginPage
  - 2.6 Cucumber — Features em português
  - 2.7 TestRunner
  - 2.8 LoginSteps
  - 2.9 UiHooks
  - 2.10 Evidências — Screenshots automáticas
- Parte 3 — Construção do Framework API
  - 3.1 REST Assured — Introdução
  - 3.2 RestClient
  - 3.3 Padrão Client-Service
  - 3.4 PostService
  - 3.5 PostRequest — Modelo POJO
  - 3.6 PostBuilder — Builder Pattern com Faker
  - 3.7 JsonUtils
  - 3.8 Payloads externalizados
  - 3.9 JSON Schema Validation
  - 3.10 Feature de API em português
  - 3.11 PostSteps
  - 3.12 TestData vs Payload
- Parte 4 — Engenharia do Framework

- 4.1 PicoContainer — Injeção de dependência
  - 4.2 ConfigReader
  - 4.3 Environment
  - 4.4 Configuração por ambiente
  - 4.5 Logging corporativo
  - 4.6 FrameworkException
  - 4.7 Screenshots configuráveis
  - 4.8 Allure Report
  - 4.9 Estratégia de Tags
  - 4.10 Organização e nomenclatura
  - Parte 5 — Integração e Manutenção
  - 5.1 GitHub Actions — Pipeline completo
  - 5.2 Como adicionar um novo teste UI
  - 5.3 Como adicionar um novo teste API
  - 5.4 Padrões de Code Review
  - 5.5 Checklist para iniciar novo projeto
  - 5.6 Troubleshooting
  - 5.7 FAQ
  - 5.8 Glossário
  - 5.9 Comandos rápidos
  - 5.10 Como evoluir a arquitetura
-

# Parte 1 — Visão Geral e Arquitetura

## 1.1 O que é automação de testes e por que automatizar

Automação de testes é a prática de usar software para executar testes que seriam feitos manualmente. Em vez de um QA clicar em cada campo, preencher formulários e verificar resultados toda vez que o sistema muda, um script faz isso de forma repetível, rápida e confiável.

A justificativa econômica é clara: um teste manual que leva 10 minutos, executado 50 vezes por sprint, consome mais de 8 horas de trabalho humano. Automatizado, ele roda em segundos a cada commit. O investimento inicial se paga em poucas semanas.

### O ciclo de vida de um bug:

Quanto mais tarde um bug é encontrado, mais caro ele é para corrigir. A automação atua na camada de prevenção — detecta regressões ANTES de chegarem à produção.

Custo de correção por fase: Desenvolvimento: 1x (barato) Testes manuais: 5x QA formal: 10x Homologação: 20x Produção: 100x (caro + dano reputacional)

A automação de testes opera na fase de "Testes" — encontra o bug quando custa 5x, não quando custa 100x.

### Quando NÃO automatizar:

Nem tudo deve ser automatizado. Automatize quando: - O teste será executado mais de 5 vezes - O fluxo é estável (não muda toda sprint) - O resultado é determinístico (mesma entrada = mesma saída) - Há ROI positivo (custo de automação < custo de execução manual)

Não automatize quando: - O fluxo muda constantemente (tela em desenvolvimento ativo) - É um teste exploratório (sem roteiro definido) - É um teste de usabilidade (requer percepção humana) - O sistema não tem interface estável (protótipos)

### Benefícios concretos:

| Aspecto                | Manual                   | Automatizado               |
|------------------------|--------------------------|----------------------------|
| Velocidade de execução | 10-30 min por fluxo      | 5-30 seg por fluxo         |
| Consistência           | Sujeito a erro humano    | Execução idêntica sempre   |
| Feedback               | Após horas/dias          | Em minutos (CI/CD)         |
| Cobertura de regressão | Parcial (tempo limitado) | Completa (todas as suítes) |
| Custo por execução     | Alto (hora-homem)        | Praticamente zero          |

|                  |          |         |
|------------------|----------|---------|
| Execução noturna | Inviável | Trivial |
|------------------|----------|---------|

**Dica:** Automatizar não significa eliminar testes manuais. Testes exploratórios, de usabilidade e cenários ad-hoc continuam sendo manuais. Automatize o que é repetitivo e regressivo.

## 1.2 O que é um framework

Há uma diferença crítica entre "ter scripts de teste" e "ter um framework de automação". Scripts soltos são arquivos Java que abrem o navegador, clicam em elementos e verificam resultados — sem padrão, sem reuso, sem estrutura.

Um framework é uma arquitetura organizada que define: - **Onde** colocar cada tipo de código (separação de responsabilidades) - **Como** elementos são localizados e interagidos (Page Objects) - **Como** dados de teste são gerenciados (configurações por ambiente) - **Como** os testes são executados e reportados (Runner + Reports) - **Como** novos testes são adicionados (padrão replicável)

| Característica          | Scripts Soltos        | Framework            |
|-------------------------|-----------------------|----------------------|
| Manutenção              | Alterar em N arquivos | Alterar em 1 lugar   |
| Reuso de código         | Copiar e colar        | Herança + composição |
| Onboarding de novos QAs | Dias                  | Horas                |
| Escala para +100 testes | Caos                  | Organizado           |
| Integração com CI/CD    | Difícil               | Nativa               |
| Reports profissionais   | Inexistente           | Allure/Cucumber      |

**Dica:** Se você precisa de mais de 1 hora para explicar como adicionar um novo teste, seu framework precisa de refatoração.

## 1.3 Pirâmide de testes

A pirâmide de testes (Martin Fowler / Mike Cohn) define a proporção ideal de testes em um sistema:

/\ / \ E2E (UI) - Poucos, lentos, frágeis /----\ / \ Integração (API) - Médios, rápidos, estáveis /-----\ / \ Unitários - Muitos, rápidos, isolados /\_\_\_\_\_\

Neste framework, cobrimos as duas camadas superiores: - **Testes de UI (E2E):** Validam fluxos completos do usuário via navegador (Selenium) - **Testes de API (Integração):** Validam contratos, status codes e regras de negócio via HTTP (REST Assured)

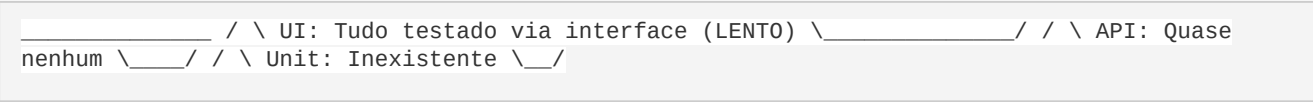
A estratégia é: manter poucos testes UI nos caminhos críticos (login, checkout) e muitos testes API para validações de dados e regras.

Características de cada camada:

| Camada    | Quantidade | Velocidade     | Estabilidade | Manutenção | Confiança  |
|-----------|------------|----------------|--------------|------------|------------|
| Unitários | Centenas   | Milissegundos  | Muito alta   | Baixa      | Isolada    |
| API       | Dezenas    | 0.5-3 segundos | Alta         | Média      | Integração |
| UI        | Poucos     | 5-30 segundos  | Média-baixa  | Alta       | E2E        |

**Distribuição recomendada:** - 70% unitários (responsabilidade do time de desenvolvimento) - 20% API (responsabilidade compartilhada QA + DEV) - 10% UI (responsabilidade principal do QA)

Anti-padrão: Pirâmide invertida (Ice Cream Cone)



Se sua organização tem mais testes UI do que API e unit, você está no anti-padrão "cone de sorvete". Migre gradualmente: para cada novo teste UI, pergunte "isso poderia ser um teste de API?".

**Dica:** Cada teste de UI que pode ser substituído por um teste de API deve ser. Testes de API são 10x mais rápidos e 5x mais estáveis.

1.4 Arquitetura em camadas

O framework segue uma arquitetura em camadas, onde cada camada tem responsabilidade única e se comunica apenas com a camada imediatamente abaixo.

[Diagrama Mermaid — visualize no arquivo .md]

Cada camada é isolada e testável independentemente. Se amanhã o Selenium for substituído por Playwright, apenas a camada de infraestrutura muda — os Steps e Features permanecem intactos.

**Dica:** A regra de ouro da arquitetura em camadas: uma camada NUNCA deve depender de uma camada acima dela. Steps conhecem Pages, mas Pages nunca importam Steps.

1.5 Fluxo completo de execução



Quando você executa `mvn test`, uma cadeia precisa de eventos acontece. Entender esse fluxo é fundamental para debugar problemas.

[Diagrama Mermaid — visualize no arquivo .md]

O ponto-chave é que o Cucumber controla o ciclo de vida. Ele lê os `.feature`, mapeia cada frase para um método Java (step definition) e gerencia os hooks de setup/teardown.

**Dica:** Se um teste falha no `@Before`, o cenário inteiro é marcado como falho. Sempre verifique os logs de hook primeiro ao investigar falhas.

## 1.6 Responsabilidade de cada camada

| Camada         | Responsabilidade                             | Exemplos de Classes                                                             |
|----------------|----------------------------------------------|---------------------------------------------------------------------------------|
| Especificação  | Descrever comportamento em linguagem natural | <code>login.feature</code> , <code>posts.feature</code>                         |
| Orquestração   | Configurar execução e ciclo de vida          | <code>TestRunner</code> , <code>UiHooks</code> , <code>ApiHooks</code>          |
| Steps          | Traduzir Gherkin para ações Java             | <code>LoginSteps</code> , <code>PostSteps</code>                                |
| Lógica (UI)    | Encapsular interações com páginas            | <code>BasePage</code> , <code>LoginPage</code>                                  |
| Lógica (API)   | Encapsular regras de negócio HTTP            | <code>PostService</code> , <code>PostBuilder</code>                             |
| Infraestrutura | Gerenciar conexões e configurações           | <code>DriverFactory</code> , <code>RestClient</code> , <code>Environment</code> |
| Utilitários    | Funções transversais de suporte              | <code>LogUtils</code> , <code>ScreenshotUtils</code> , <code>JsonUtils</code>   |
| Modelos        | Representar dados estruturados               | <code>PostRequest</code>                                                        |
| Exceções       | Erros semânticos do framework                | <code>FrameworkException</code>                                                 |

**Dica:** Se uma classe não se encaixa claramente em nenhuma camada, provavelmente ela está fazendo coisas demais e deve ser dividida.

## 1.7 Decisões tecnológicas e justificativas

| Tecnologia | Versão | Justificativa |
|------------|--------|---------------|
|------------|--------|---------------|

|                    |                 |                                                          |
|--------------------|-----------------|----------------------------------------------------------|
| Java               | 8               | Compatibilidade máxima com ambientes corporativos; LTS   |
| Maven              | 3.x             | Gerenciador de build padrão; integração nativa com CI    |
| Selenium WebDriver | 3.141.59        | Última versão compatível com Java 8; estável em produção |
| Cucumber           | 7.18.0          | BDD em português; integração madura com JUnit            |
| JUnit 4            | 4.13.2          | Runner padrão para Cucumber; compatível com Surefire     |
| REST Assured       | 4.5.1           | Última versão Java 8; API fluente given/when/then        |
| PicoContainer      | 7.18.0          | DI leve por cenário; zero configuração                   |
| Allure Report      | 2.24.0          | Relatórios visuais ricos; histórico de execuções         |
| SLF4J + Logback    | 1.7.36 / 1.2.12 | Logging desacoplado; configurável por ambiente           |
| JavaFaker          | 1.0.2           | Dados dinâmicos; evita dados fixos em testes             |
| AspectJ            | 1.9.19          | Necessário para integração Allure (weaving)              |
| GitHub Actions     | -               | CI/CD gratuito; integração nativa com repositório        |

**Dica:** A escolha do Java 8 é estratégica: a maioria dos ambientes corporativos brasileiros ainda roda Java 8 em produção. Seu framework de testes deve ser compatível com o ambiente onde será executado.

## 1.8 Estrutura completa de diretórios

```
selenium-cucumber-project/
├── .github/
│   └── workflows/
│       └── testes.yml # Pipeline CI/CD
├── src/
│   └── test/
│       ├── java/
│       │   ├── api/
│       │   ├── builders/
│       │   ├── PostBuilder.java # Builder Pattern + Faker
│       │   ├── clients/
│       │   │   ├── RestClient.java # Cliente HTTP corporativo
│       │   ├── models/
│       │   │   ├── PostRequest.java # POJO de request
│       │   ├── services/
│       │   │   ├── PostService.java # Lógica de negócio API
│       │   ├── config/
│       │   │   ├── ConfigReader.java # Leitura de .properties
│       │   │   ├── Environment.java # Gerenciamento de ambientes
│       │   ├── drivers/
│       │   │   ├── DriverFactory.java # Criação do
```

```
navegador ■ ■ ■ ■ ■ DriverManager.java # ThreadLocal para paralelo ■ ■ ■ ■ exceptions/ ■
■ ■ ■ ■ FrameworkException.java # Exceção customizada ■ ■ ■ ■ hooks/ ■ ■ ■ ■ ■
ApiHooks.java # Hooks para @api ■ ■ ■ ■ ■ UiHooks.java # Hooks para @ui ■ ■ ■ ■ ■ pages/ ■
■ ■ ■ ■ ■ base/ ■ ■ ■ ■ ■ BasePage.java # Classe abstrata base ■ ■ ■ ■ ■ login/ ■ ■ ■
■ ■ ■ ■ LoginPage.java # Page Object do login ■ ■ ■ ■ ■ runners/ ■ ■ ■ ■ ■ TestRunner.java #
Ponto de entrada ■ ■ ■ ■ ■ steps/ ■ ■ ■ ■ ■ api/ ■ ■ ■ ■ ■ PostSteps.java # Steps de API
■ ■ ■ ■ ■ ui/ ■ ■ ■ ■ ■ LoginSteps.java # Steps de UI ■ ■ ■ ■ ■ úteis/ ■ ■ ■ ■ ■
JsonUtils.java # Carregamento de JSON ■ ■ ■ ■ ■ LogUtils.java # Logging corporativo ■ ■ ■ ■ ■
ScreenshotUtils.java # Captura de tela ■ ■ ■ ■ ■ resources/ ■ ■ ■ ■ ■ environments/ ■ ■ ■ ■ ■
dev.properties # Configuração DEV ■ ■ ■ ■ ■ hml.properties # Configuração HML ■ ■ ■ ■ ■ features/
■ ■ ■ ■ ■ api/ ■ ■ ■ ■ ■ posts.feature # Cenários de API ■ ■ ■ ■ ■ ui/ ■ ■ ■ ■ ■
login.feature # Cenários de UI ■ ■ ■ ■ ■ payloads/ ■ ■ ■ ■ ■ posts/ ■ ■ ■ ■ ■ create-post.json #
Payload de criação ■ ■ ■ ■ ■ update-post.json # Payload de atualização ■ ■ ■ ■ ■ schemas/ ■ ■
■ ■ ■ ■ post-schema.json # Schema de validação ■ ■ ■ ■ ■ logback.xml # Configuração de logging ■ ■ ■
pom.xml # Configuração Maven
```

A organização segue o princípio de **coesão**: arquivos que mudam juntos ficam juntos. Todas as classes de API ficam em `api/`, todos os resources de API ficam em subpastas correspondentes.

**Dica:** Ao criar um novo recurso (ex: Users), replique a mesma estrutura: `api/services/UserService.java`, `api/models/UserRequest.java`, `payloads/users/create-user.json`, `features/api/users.feature`.

## 1.9 Pré-requisitos

Para executar este framework localmente, você precisa de quatro componentes fundamentais. Cada um cumpre um papel específico na cadeia de execução.

### Visão geral dos componentes:

```
##### JDK 8 #####> Maven
#####> ChromeDriver #####> Chrome ## (compila)## (executa)## (controla) ##
##(renderiza)## #####
```

- **JDK 8** compila o código Java do framework
- **Maven** gerencia dependências e executa os testes
- **ChromeDriver** é a ponte entre o Selenium e o Chrome
- **Chrome** é o navegador que renderiza as páginas

## 1. JDK 8 (Java Development Kit)

O JDK inclui o compilador (`javac`) e a JVM (`java`). Certifique-se de instalar o JDK (não apenas o JRE).

Verifique se já está instalado:

```
java -version # Esperado: java version "1.8.x" javac -version # Esperado: javac 1.8.x
```

Caso não tenha, instale o Temurin (AdoptOpenJDK): - Windows: Baixe em <https://adoptium.net/> e escolha JDK 8 LTS - Configure `JAVA_HOME` apontando para a pasta de instalação - Adicione `%JAVA_HOME%\bin` ao PATH - Reinicie o terminal após configurar

## Verificação pós-instalação:

```
echo %JAVA_HOME% # Esperado: C:\Program Files\Eclipse Adoptium\jdk-8.x.x-hotspot (ou similar)
```

## 2. Apache Maven 3.x

O Maven gerencia dependências e o ciclo de build. Ele baixa automaticamente todas as bibliotecas declaradas no pom.xml.

```
mvn -version # Esperado: Apache Maven 3.x.x # Esperado: Java version: 1.8.x (confirma que usa o JDK correto)
```

- Windows: Baixe em <https://maven.apache.org/download.cgi>
- Extraia para C:\maven
- Configure `MAVEN_HOME` e adicione `%MAVEN_HOME%\bin` ao PATH

**Primeiro uso:** Na primeira execução, o Maven baixa todas as dependências (~100 MB). As execuções seguintes são rápidas porque usa cache local (`~/.m2/repository/`).

## 3. Google Chrome (última versão)

O framework usa Chrome como navegador padrão. Mantenha-o atualizado para evitar incompatibilidades com o ChromeDriver.

Para verificar a versão: acesse `chrome://version` na barra de endereço.

## 4. ChromeDriver

O ChromeDriver deve ser compatível com a versão do Chrome instalada: - Verifique a versão do Chrome em `chrome://version` - Baixe o driver correspondente em <https://chromedriver.chromium.org/downloads> - Para Chrome 115+: <https://googlechromelabs.github.io/chrome-for-testing/> - Coloque em um diretório acessível (ex: `C:\chromedriver\`) - Configure a variável de ambiente `CHROME_DRIVER_PATH` ou use o path padrão

### Compatibilidade Chrome/ChromeDriver:

| Chrome | ChromeDriver       |
|--------|--------------------|
| 120.x  | chromedriver 120.x |
| 121.x  | chromedriver 121.x |
| 125.x  | chromedriver 125.x |

A regra é simples: a versão MAJOR (primeiro número) deve ser idêntica.

### Validação completa do ambiente:

```
java -version && mvn -version && google-chrome --version && chromedriver --version
```

Se todos os quatro comandos retornam versões, seu ambiente está pronto.

**Dica:** Em ambientes CI (GitHub Actions), o Chrome e o ChromeDriver são instalados automaticamente pelo pipeline. A configuração local é necessária apenas para desenvolvimento.

---

## Parte 2 — Construção do Framework UI

### 2.1 Criação do projeto Maven

O `pom.xml` é o coração de qualquer projeto Java gerenciado pelo Maven. Ele define as dependências, plugins de build e configurações de compilação. Cada dependência foi escolhida com critério de compatibilidade com Java 8 e maturidade em produção.

```
<?xml version="1.0" encoding="UTF-8"?> <project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd"> <modelVersion>4.0.0</modelVersion>
<groupId>com.automação</groupId>
<artifactId>selenium-restassured-cucumber-github-actions</artifactId>
<version>1.0.0</version> <packaging>jar</packaging>
<name>selenium-restassured-cucumber-github-actions</name> <description>Automação de testes
Web (Selenium) e API (REST Assured) com Cucumber BDD</description> <properties>
<java.version>8</java.version> <maven.compiler.source>${java.version}</maven.compiler.source>
<maven.compiler.target>${java.version}</maven.compiler.target>
<project.build.sourceEncoding>UTF-8</project.build.sourceEncoding> <!-- Versões das
dependências --> <!-- Selenium 3.x – última versão compatível com Java 8 -->
<selenium.version>3.141.59</selenium.version> <!-- Cucumber 7.x requer Java 8+ -->
<cucumber.version>7.18.0</cucumber.version> <junit.version>4.13.2</junit.version> <!-- REST
Assured 4.x – última versão com suporte a Java 8 -->
<rest.assured.version>4.5.1</rest.assured.version> <!-- Allure Report -->
<allure.version>2.24.0</allure.version> </properties> <dependencies> <!-- Selenium WebDriver
--> <dependency> <groupId>org.seleniumhq.selenium</groupId>
<artifactId>selenium-java</artifactId> <version>${selenium.version}</version> </dependency>
<!-- Cucumber JVM --> <dependency> <groupId>io.cucumber</groupId>
<artifactId>cucumber-java</artifactId> <version>${cucumber.version}</version> </dependency>
<!-- Cucumber + JUnit integration --> <dependency> <groupId>io.cucumber</groupId>
<artifactId>cucumber-junit</artifactId> <version>${cucumber.version}</version>
<scope>test</scope> </dependency> <!-- JUnit --> <dependency> <groupId>junit</groupId>
<artifactId>junit</artifactId> <version>${junit.version}</version> <scope>test</scope>
</dependency> <!-- REST Assured - testes de API REST --> <dependency>
<groupId>io.rest-assured</groupId> <artifactId>rest-assured</artifactId>
<version>${rest.assured.version}</version> <scope>test</scope> </dependency> <!-- JSON Schema
Validator - valida estrutura da resposta --> <dependency> <groupId>io.rest-assured</groupId>
<artifactId>json-schema-validator</artifactId> <version>${rest.assured.version}</version>
<scope>test</scope> </dependency> <!-- Allure Cucumber Integration --> <dependency>
<groupId>io.qameta.allure</groupId> <artifactId>allure-cucumber7-jvm</artifactId>
<version>${allure.version}</version> <scope>test</scope> <exclusions> <exclusion>
<groupId>io.cucumber</groupId> <artifactId>gherkin</artifactId> </exclusion> <exclusion>
<groupId>io.cucumber</groupId> <artifactId>messages</artifactId> </exclusion> </exclusions>
</dependency> <!-- AspectJ Weaver (necessário para Allure) --> <dependency>
<groupId>org.aspectj</groupId> <artifactId>aspectjweaver</artifactId>
<version>1.9.19</version> <scope>test</scope> </dependency> <!-- PicoContainer – Injeção de
Dependência por cenário no Cucumber --> <dependency> <groupId>io.cucumber</groupId>
<artifactId>cucumber-picocontainer</artifactId> <version>${cucumber.version}</version>
<scope>test</scope> </dependency> <!-- SLF4J API – fachada de logging --> <dependency>
<groupId>org.slf4j</groupId> <artifactId>slf4j-api</artifactId> <version>1.7.36</version>
</dependency> <!-- Logback – implementação SLF4J --> <dependency>
<groupId>ch.qos.logback</groupId> <artifactId>logback-classic</artifactId>
<version>1.2.12</version> </dependency> <!-- JavaFaker – geração de dados de teste dinâmicos
--> <dependency> <groupId>com.github.javafaker</groupId> <artifactId>javafaker</artifactId>
<version>1.0.2</version> <scope>test</scope> </dependency> </dependencies> <build> <plugins>
<!-- Maven Surefire Plugin - executa os testes JUnit --> <plugin>
<groupId>org.apache.maven.plugins</groupId> <artifactId>maven-surefire-plugin</artifactId>
<version>3.2.5</version> <configuration> <includes> <include>*/TestRunner.java</include>
</includes> <argLine> -Djavax.net.ssl.trustStore=${user.home}/.maven-cacerts
-Djavax.net.ssl.trustStorePassword=changeit
-javaagent:"${settings.localRepository}/org/aspectj/aspectjweaver/1.9.19/aspectjweaver-1.9.19.jar"
</argLine> <systemPropertyVariables>
```

```
<allure.results.directory>target/allure-results</allure.results.directory>
</systemPropertyVariables> </configuration> </plugin> <!-- Maven Compiler Plugin --> <plugin>
<groupId>org.apache.maven.plugins</groupId> <artifactId>maven-compiler-plugin</artifactId>
<version>3.13.0</version> <configuration> <source>${java.version}</source>
<target>${java.version}</target> <encoding>UTF-8</encoding> </configuration> </plugin> <!--
Allure Maven Plugin - gera relatório --> <plugin> <groupId>io.qameta.allure</groupId>
<artifactId>allure-maven</artifactId> <version>2.12.0</version> <configuration>
<reportVersion>${allure.version}</reportVersion>
<resultsDirectory>allure-results</resultsDirectory> </configuration> </plugin> </plugins>
</build> </project>
```

### Decisões de design do pom.xml:

- Todas as versões estão centralizadas em `<properties>` para facilitar atualizações futuras — basta alterar um número em um lugar.
- O Surefire está configurado para encontrar apenas `TestRunner.java`, garantindo que o Cucumber controle a execução (e não o JUnit direto).
- O `argLine` do Surefire injeta o AspectJ como agente JVM, necessário para que o Allure intercepte os steps automaticamente.
- As exclusões no `allure-cucumber7-jvm` evitam conflitos de versão com as bibliotecas Gherkin que já vêm com o Cucumber.

**Dica:** Sempre defina versões como *properties*. Quando precisar atualizar o Cucumber de 7.18 para 7.19, você muda em UM lugar e todas as dependências se ajustam.

## 2.2 DriverFactory

A DriverFactory é responsável por criar instâncias do WebDriver. Ela encapsula toda a lógica de configuração do navegador, incluindo a detecção automática de ambiente CI para modo headless.

```
package drivers; import org.openqa.selenium.WebDriver; import
org.openqa.selenium.chrome.ChromeDriver; import org.openqa.selenium.chrome.ChromeOptions;
import utils.LogUtils; /** * Cria instâncias de WebDriver. */ public class DriverFactory {
private static final boolean IN_CI = System.getenv("CI") != null ||
System.getenv("JENKINS_URL") != null; public WebDriver create(String browser) {
LogUtils.info("Criando driver: " + browser + (IN_CI ? " [headless]" : " [visual]")); switch
(browser.toLowerCase()) { case "chrome": return createChrome(); default: throw new
IllegalArgumentException("Browser não suportado: " + browser); } } private WebDriver
createChrome() { if (!IN_CI) { String path = System.getenv("CHROME_DRIVER_PATH") != null ?
System.getenv("CHROME_DRIVER_PATH") :
"C:\\chromedriver\\chromedriver-win64\\chromedriver.exe";
System.setProperty("webdriver.chrome.driver", path); } ChromeOptions options = new
ChromeOptions(); if (IN_CI) { options.addArguments("--headless=new", "--no-sandbox",
"--disable-dev-shm-usage", "--window-size=1920,1080"); } else {
options.addArguments("--start-maximized"); } options.addArguments("--disable-notifications",
"--remote-allow-origins=*"); return new ChromeDriver(options); } }
```

### Decisões de design:

- A detecção de CI usa variáveis de ambiente padrão (`CI` para GitHub Actions, `JENKINS_URL` para Jenkins). Isso permite que o mesmo código rode local e no pipeline sem nenhuma alteração.

- Em CI, o Chrome roda headless com `--no-sandbox` e `--disable-dev-shm-usage` — flags obrigatórias para containers Linux.
- O `--window-size=1920,1080` garante que screenshots em CI tenham resolução consistente.
- Localmente, o path do ChromeDriver aceita variável de ambiente ou usa um padrão sensato.

**Dica:** Nunca faça commit de caminhos absolutos locais (ex: `C:\Users\joao\...`). Use variáveis de ambiente para paths que variam entre máquinas.

---

## 2.3 DriverManager

O DriverManager resolve um problema crítico: como compartilhar a mesma instância de WebDriver entre Hooks, Steps e Pages sem acoplá-los via campo estático comum. A resposta é ThreadLocal.

```
package drivers; import org.openqa.selenium.WebDriver; /** * Gerencia o WebDriver via ThreadLocal (seguro para paralelo). */ public class DriverManager { private static final ThreadLocal<WebDriver> driver = new ThreadLocal<>(); private DriverManager() {} public static WebDriver getDriver() { return driver.get(); } public static void setDriver(WebDriver webDriver) { driver.set(webDriver); } public static void quit() { WebDriver d = driver.get(); if (d != null) { d.quit(); driver.remove(); } } }
```

### Decisões de design:

- `ThreadLocal<WebDriver>` garante que cada thread tem sua própria instância do navegador. Quando o Cucumber executar cenários em paralelo (futuro), cada cenário terá seu Chrome isolado sem interferência.
- O construtor é privado (padrão utility class) porque todos os métodos são estáticos.
- O `quit()` faz `driver.remove()` após fechar — isso previne memory leaks em execuções longas com muitos cenários.

**Dica:** `ThreadLocal` é essencial para paralelismo, mas também é útil em execução serial: ele garante um ponto único de acesso ao driver sem precisar passar o objeto por parâmetro em toda a cadeia.

---

## 2.4 BasePage

A BasePage é a classe abstrata que serve de fundação para todos os Page Objects. Ela encapsula as interações comuns com o Selenium (clicar, digitar, navegar) e configura o WebDriverWait com timeout do ambiente.

```
package pages.base; import config.Environment; import org.openqa.selenium.By; import org.openqa.selenium.TimeoutException; import org.openqa.selenium.WebDriver; import org.openqa.selenium.WebElement; import org.openqa.selenium.support.ui.ExpectedConditions; import org.openqa.selenium.support.ui.WebDriverWait; import utils.LogUtils; /** * Classe base para todos os Page Objects. */ public abstract class BasePage { protected final WebDriver driver; protected final WebDriverWait wait; protected BasePage(WebDriver driver) { this.driver = driver; int timeout = new Environment().getInt("timeout.explicit", 10); this.wait = new WebDriverWait(driver, timeout); } protected void navigate(String url) {
```



```
LogUtils.info("Navegando: " + url); driver.get(url); } protected void type(By locator, String text) { WebElement element = wait.until(ExpectedConditions.visibilityOfElementLocated(locator)); element.clear(); element.sendKeys(text); } protected void click(By locator) { wait.until(ExpectedConditions.elementToBeClickable(locator)).click(); } protected String getText(By locator) { return wait.until(ExpectedConditions.visibilityOfElementLocated(locator)).getText(); } protected boolean urlContains(String fragment) { try { return wait.until(ExpectedConditions.urlContains(fragment)); } catch (TimeoutException e) { LogUtils.warn("Timeout aguardando URL conter: " + fragment); return false; } } }
```

### Decisões de design:

- A classe é `abstract` porque nunca deve ser instanciada diretamente — ela só existe para ser herdada por Pages concretas.
- Os métodos são `protected` (não `public`) porque são destinados apenas às subclasses, não ao mundo externo. Steps nunca devem chamar `type()` diretamente.
- O timeout vem do `Environment`, então cada ambiente pode ter timeouts diferentes (DEV = 10s, HML = 15s).
- Todo `type()` faz `clear()` antes de `sendKeys()` para evitar concatenação de texto em campos pré-preenchidos.
- O `urlContains()` captura `TimeoutException` e retorna `false` em vez de propagar a exceção — isso torna as asserções mais limpas nos Steps.

**Dica:** Se você precisa de uma interação específica que não existe na `BasePage` (ex: `drag-and-drop`), adicione-a aqui. Todas as Pages herdam automaticamente.

## 2.5 LoginPage

O `LoginPage` implementa o Page Object Pattern: cada página da aplicação tem uma classe Java correspondente que encapsula seus elementos e ações. O código externo (Steps) nunca lida com localizadores — apenas com métodos semânticos.

```
package pages.login; import org.openqa.selenium.By; import org.openqa.selenium.WebDriver; import pages.base.BasePage; /** * Page Object da página de Login. */ public class LoginPage extends BasePage { private final By usernameField = By.name("username"); private final By passwordField = By.name("password"); private final By loginButton = By.cssSelector("button[type='submit']"); private final By errorMessage = By.cssSelector(".oxd-alert-content-text"); public LoginPage(WebDriver driver) { super(driver); } public void open(String url) { navigate(url); } public void fillUsername(String username) { type(usernameField, username); } public void fillPassword(String password) { type(passwordField, password); } public void clickLogin() { click(loginButton); } public String getErrorMessage() { return getText(errorMessage); } public boolean isOnDashboard() { return urlContains("/dashboard"); } }
```

### Decisões de design:

- Localizadores são `private final` — eles pertencem exclusivamente a esta Page e nunca mudam após construção.

- Métodos têm nomes semânticos (`fillUsername`, `clickLogin`) em vez de técnicos (`typeInField1`, `clickButton`). Isso torna os Steps legíveis como documentação.
- A Page recebe o driver no construtor (injeção de dependência) em vez de buscar de um singleton. Isso facilita testes unitários da própria Page.
- Não há lógica de negócio na Page: ela não decide se o login foi bem-sucedido — apenas expõe `isOnDashboard()` para o Step decidir.

**Dica:** Um Page Object nunca deve conter asserções (Assert). A responsabilidade de verificar é do Step. A Page apenas informa o estado atual.

## 2.6 Cucumber — Features em português

O Cucumber permite escrever especificações executáveis em linguagem natural. Com a diretiva `# language: pt`, todas as keywords ficam em português, tornando os cenários acessíveis para analistas de negócio e POs.

```
# language: pt @ui Funcionalidade: Login no sistema Como um usuário registrado Quero fazer login na aplicação Para acessar as funcionalidades do sistema Contexto: Dado que estou na página de login @smoke Cenário: Login com credenciais válidas Quando faço login como administrador Então devo ser redirecionado para a página inicial Cenário: Login com senha incorreta Quando faço login com usuário "admin" e senha incorreta Então devo ver a mensagem de erro "Invalid credentials" Esquema do Cenário: Login com credenciais inválidas Quando faço login com usuário "<usuario>" e senha "<senha>" Então devo ver a mensagem de erro "<mensagem>" Exemplos: | usuario | senha | mensagem | | usuarioErrado | admin123 | Invalid credentials | | wronguser | wrongpass | Invalid credentials |
```

### Tabela de keywords Gherkin em português:

| Português          | Inglês           | Função                                 |
|--------------------|------------------|----------------------------------------|
| Funcionalidade     | Feature          | Agrupa cenários relacionados           |
| Contexto           | Background       | Steps executados antes de cada cenário |
| Cenário            | Scenário         | Caso de teste individual               |
| Esquema do Cenário | Scenário Outline | Template com múltiplas combinações     |
| Exemplos           | Examples         | Tabela de dados para Outline           |
| Dado               | Given            | Pré-condição (setup)                   |
| Quando             | When             | Ação do usuário                        |
| Então              | Then             | Resultado esperado                     |

### Decisões de design:

- A tag `@ui` permite filtrar apenas testes de interface. A tag `@smoke` marca cenários críticos para execução rápida.
- O `Contexto` (Background) evita repetir "Dado que estou na página de login" em todo cenário.
- O `Esquema do Cenário com Exemplos` é um teste parametrizado — executa o mesmo fluxo com dados diferentes.

**Dica:** Features devem ser escritas na perspectiva do usuário, não do sistema. Use "Quando faço login" (usuário) em vez de "Quando o sistema valida credenciais" (técnico).

## 2.7 TestRunner

O TestRunner é o ponto de entrada que conecta Maven, JUnit e Cucumber. Ele define onde estão as features, onde estão os steps e quais plugins de report serão usados.

```
package runners; import io.cucumber.junit.Cucumber; import io.cucumber.junit.CucumberOptions;
import org.junit.runner.RunWith; /** * Runner principal. * * mvn test -> todos * mvn test
-Dcucumber.filter.tags="@smoke" -> smoke * mvn test -Dcucumber.filter.tags="@api" -> API *
mvn test -Dcucumber.filter.tags="@ui" -> UI * mvn test -Denvironment=hml -> ambiente HML *
mvn allure:serve -> relatório */ @RunWith(Cucumber.class) @CucumberOptions( features =
"src/test/resources/features", glue = {"steps", "hooks"}, plugin = { "pretty",
"html:target/cucumber-reports/cucumber.html", "json:target/cucumber-reports/cucumber.json",
"junit:target/cucumber-reports/cucumber.xml",
"io.qameta.allure.cucumber7jvm.AllureCucumber7Jvm" }, monochrome = true ) public class
TestRunner { }
```

### Decisões de design:

- `features` aponta para a pasta raiz — o Cucumber escaneia recursivamente todas as subpastas (`ui/`, `api/`).
- `glue` inclui tanto `steps` quanto `hooks`. Se você esquecer de incluir `hooks`, os `@Before/@After` não serão executados.
- Os quatro plugins geram: saída colorida no console (`pretty`), relatório HTML nativo, JSON para integração com ferramentas, XML para JUnit report e dados para o Allure.
- `monochrome = true` remove caracteres de escape ANSI do output — útil em logs de CI.
- A filtragem por tags é feita via linha de comando (`-Dcucumber.filter.tags`), não no Runner. Isso permite flexibilidade sem recompilar.

**Dica:** Nunca coloque tags fixas no `@CucumberOptions` em produção. Isso impediria a execução de todos os testes. Use `-Dcucumber.filter.tags` no comando Maven.

## 2.8 LoginSteps

Os Step Definitions conectam a linguagem natural do Gherkin com código Java executável. Cada frase "Dado/Quando/Então" mapeia para um método anotado. O PicoContainer injeta dependências automaticamente via construtor.

```
package steps.ui; import config.Environment; import drivers.DriverManager; import
io.cucumber.java.pt.Dado; import io.cucumber.java.pt.Então; import
io.cucumber.java.pt.Quando; import org.junit.Assert; import pages.login.LoginPage; /** *
Steps de Login (UI). * Environment injetado via PicoContainer. */ public class LoginSteps {
private final Environment env; private LoginPage loginPage; public LoginSteps(Environment
env) { this.env = env; } @Dado("que estou na página de login") public void openLogin() {
loginPage = new LoginPage(DriverManager.getDriver()); loginPage.open(env.baseUrl); }
@Quando("faço login como administrador") public void loginAsAdmin() {
loginPage.fillUsername(env.get("usuario.admin"));
loginPage.fillPassword(env.get("senha.admin")); loginPage.clickLogin(); } @Quando("faço login
com usuário {string} e senha {string}") public void loginWith(String user, String pass) {
loginPage.fillUsername(user); loginPage.fillPassword(pass); loginPage.clickLogin(); }
@Quando("faço login com usuário {string} e senha incorreta") public void
loginWithWrongPassword(String user) { loginPage.fillUsername(user);
loginPage.fillPassword(env.get("senha.invalida")); loginPage.clickLogin(); } @Então("devo ser
redirecionado para a página inicial") public void shouldBeOnDashboard() {
Assert.assertTrue("Não redirecionou para o dashboard", loginPage.isOnDashboard()); }
@Então("devo ver a mensagem de erro {string}") public void shouldSeeError(String expected) {
Assert.assertEquals("Mensagem incorreta", expected, loginPage.getErrorMessage()); } }
```

### Decisões de design:

- O `Environment` é injetado via construtor pelo `PicoContainer`. Nenhuma configuração XML ou anotação adicional é necessária — basta declarar no construtor.
- Credenciais vêm do arquivo de propriedades (`env.get("usuario.admin")`), nunca hardcoded no Step. Isso permite trocar credenciais por ambiente sem alterar código.
- O `LoginPage` é instanciado no step `@Dado` (não no construtor) porque o driver só existe após o Hook abrir o navegador.
- Steps usam `{string}` como expression do Cucumber para capturar parâmetros dinâmicos do Gherkin.

**Dica:** Cada classe de Steps deve ser coesa — trate apenas de um assunto. `LoginSteps` cuida de login, não de cadastro. Se a feature crescer, crie novas classes de Steps.

## 2.9 UiHooks

Os Hooks controlam o ciclo de vida do navegador: abrir antes de cada cenário UI e fechar depois. Eles também gerenciam a captura de evidências (screenshots).

```
package hooks; import config.Environment; import drivers.DriverFactory; import
drivers.DriverManager; import io.cucumber.java.After; import io.cucumber.java.Before; import
io.cucumber.java.Scenário; import org.openqa.selenium.WebDriver; import utils.LogUtils;
import utils.ScreenshotUtils; import java.util.concurrent.TimeUnit; /** * Hooks para cenários
@ui. */ public class UiHooks { private final Environment env; public UiHooks() { this.env =
new Environment(); } @Before(value = "@ui", order = 0) public void logScenário(Scenário
scenário) { LogUtils.info("=== [UI] " + cenário.getName() + " ==="); } @Before(value =
```

```
"@ui", order = 1) public void openBrowser() { if (DriverManager.getDriver() == null) { String browser = env.get("browser", "chrome"); int implicitWait = env.getInt("timeout.implicit", 10); int pageLoad = env.getInt("timeout.pageLoad", 30); DriverFactory factory = new DriverFactory(); WebDriver driver = factory.create(browser); driver.manage().timeouts().implicitlyWait(implicitWait, TimeUnit.SECONDS); driver.manage().timeouts().pageLoadTimeout(pageLoad, TimeUnit.SECONDS); DriverManager.setDriver(driver); } } @After(value = "@ui") public void closeBrowser(Scenário cenário) { WebDriver driver = DriverManager.getDriver(); if (driver == null) return; String mode = env.get("screenshot.mode", "failure_only"); boolean shouldCapture = "always".equals(mode) || cenário.isFailed(); if (shouldCapture) { byte[] screenshot = ScreenshotUtils.capture(driver); if (screenshot.length > 0) { String status = cenário.isFailed() ? "FALHA" : "SUCESSO"; cenário.attach(screenshot, "image/png", status + " - " + cenário.getName()); LogUtils.info("Screenshot [" + status + "]"); } } DriverManager.quit(); LogUtils.info("=== Navegador encerrado ==="); } }
```

## Ciclo de vida do cenário UI:

[Diagrama Mermaid — visualize no arquivo .md]

## Decisões de design:

- `value = "@ui"` faz o hook rodar APENAS em cenários com a tag `@ui`. Cenários de API não abrem navegador.
- `order = 0` e `order = 1` garantem sequência: primeiro loga, depois abre o browser. Ordens menores executam primeiro.
- O `if (DriverManager.getDriver() == null)` previne reabrir o navegador se ele já estiver aberto (defensivo).
- O `screenshot mode` é configurável por ambiente: em DEV capture apenas falhas (economiza tempo), em HML capture sempre (mais evidências).
- `cenário.attach()` anexa a screenshot diretamente ao relatório Cucumber/Allure.

**Dica:** O `@After` sempre executa, mesmo se o cenário falhou. Por isso ele é o local perfeito para *cleanup* (fechar browser, limpar dados). Nunca faça *cleanup* no último step.

## 2.10 Evidências — Screenshots automáticas

A captura de evidências é essencial para rastreabilidade. O `ScreenshotUtils` encapsula a lógica de captura, enquanto o `UiHooks` decide QUANDO capturar com base na configuração.

```
package utils; import org.openqa.selenium.OutputType; import org.openqa.selenium.TakesScreenshot; import org.openqa.selenium.WebDriver; /** * Captura de screenshots. */ public class ScreenshotUtils { private ScreenshotUtils() {} public static byte[] capture(WebDriver driver) { if (driver instanceof TakesScreenshot) { return ((TakesScreenshot) driver).getScreenshotAs(OutputType.BYTES); } return new byte[0]; } }
```

## Modos de captura:

| Modo | Configuração | Quando captura | Uso recomendado |
|------|--------------|----------------|-----------------|
|------|--------------|----------------|-----------------|

|                           |                                           |                                |                       |
|---------------------------|-------------------------------------------|--------------------------------|-----------------------|
| <code>failure_only</code> | <code>screenshot.mode=failure_only</code> | Apenas quando cenário<br>falha | DEV (economiza tempo) |
| <code>always</code>       | <code>screenshot.mode=always</code>       | Sempre (sucesso e falha)       | HML (mais evidências) |

A decisão de modo fica no arquivo de propriedades do ambiente. O código em UiHooks avalia:

```
String mode = env.get("screenshot.mode", "failure_only"); boolean shouldCapture =  
"always".equals(mode) || cenário.isFailed();
```

A screenshot é retornada como `byte[]` e anexada ao cenário Cucumber via `cenário.attach()`. Isso garante que a imagem aparece tanto no relatório Cucumber HTML quanto no Allure Report.

**Dica:** Em ambientes de homologação, use *always* — isso gera evidências visuais para auditoria mesmo quando os testes passam.

## Parte 3 — Construção do Framework API

### 3.1 REST Assured — Introdução

REST Assured é uma biblioteca Java para testar APIs RESTful de forma fluente. Ela usa a mesma estrutura Given/When/Then do BDD para construir requisições HTTP e validar respostas.

### Conceitos fundamentais:

```
given() // Configuração (headers, body, auth) .baseUrl("https://api.example.com")
.contentType(ContentType.JSON) .body(payload) .when() // Ação (verbo HTTP) .post("/users")
.then() // Verificação (assertões) .statusCode(201) .body("id", notNullValue());
```

## Anatomia de uma chamada REST:

Uma chamada de API é composta por:

[illegible]

Nos testes, validamos: 1. **Status Code** — a API retornou o código esperado? 2. **Headers** — o Content-Type está correto? 3. **Body** — os campos têm os valores esperados? 4. **Schema** — a estrutura da resposta está correta?

## Por que REST Assured em vez de HttpClient puro:

| Aspecto              | HttpClient                    | REST Assured                |
|----------------------|-------------------------------|-----------------------------|
| Sintaxe              | Verbosa (20+ linhas)          | Fluente (5 linhas)          |
| Validações           | Manuais (parse JSON + assert) | Built-in (jsonPath, schema) |
| Logging              | Manual                        | Automático (log().all())    |
| Integração BDD       | Nenhuma                       | Nativa (given/when/then)    |
| Curva de aprendizado | Alta                          | Baixa                       |
| Serialização JSON    | Manual                        | Automática                  |
| Cookie handling      | Manual                        | Automático                  |

### Exemplo comparativo:

HttpClient (verboso):

```
HttpClient client = HttpClient.newHttpClient(); HttpRequest request =
HttpRequest.newBuilder().uri(URI.create("https://api.example.com/posts/1"))
.header("Accept", "application/json").GET().build(); HttpResponse<String> response =
client.send(request, BodyHandlers.ofString()); JSONObject json = new
JSONObject(response.body()); assertEquals(200, response.statusCode()); assertEquals(1,
json.getInt("id"));
```

REST Assured (fluente):

```
given().baseUrl("https://api.example.com").when().get("/posts/1").then().statusCode(200)
.body("id", equalTo(1));
```

O REST Assured reduz o código de teste em ~70% mantendo a mesma cobertura.

Neste framework, o REST Assured é encapsulado dentro do `RestClient` para adicionar features corporativas (logging, Allure attachments, reuso de configuração).

**Dica:** REST Assured foi projetado para TESTES, não para produção. Nunca use em código de aplicação — use `HttpClient` ou `OkHttp` para isso.

## 3.2 RestClient

O `RestClient` é o cliente HTTP corporativo do framework. Ele encapsula o REST Assured adicionando logging automático, attachments para o Allure e uma interface simplificada para os Services.

```
package api.clients; import io.qameta.allure.Allure; import io.restassured.http.ContentType;
import io.restassured.response.Response; import
io.restassured.specification.RequestSpecification; import utils.LogUtils; import static
io.restassured.RestAssured.given; /** * Cliente HTTP corporativo. * Instância por cenário
(thread-safe via PicoContainer). * Anexa request/response ao Allure automaticamente. */
public class RestClient { private Response response; private RequestSpecification request;
private String baseUrl; private String lastBody; public RestClient() {} public void
setBaseUrl(String baseUrl) { this.baseUrl = baseUrl; this.request = given().baseUrl(baseUrl)
.contentType(ContentType.JSON).accept(ContentType.JSON); } public void addHeader(String key,
String value) { request = request.header(key, value); } public void setBody(String body) {
this.lastBody = body; request = request.body(body); } public void get(String endpoint) {
execute("GET", endpoint); } public void post(String endpoint) { execute("POST", endpoint); }
public void put(String endpoint) { execute("PUT", endpoint); } public void delete(String
endpoint) { execute("DELETE", endpoint); } public int getStatusCode() { return
response.getStatusCode(); } public String getContentType() { return
response.getContentType(); } public Response getResponse() { return response; } public String
getResponseBody() { return response.getBody().asString(); } private void execute(String
method, String endpoint) { LogUtils.info(method + " " + baseUrl + endpoint); switch (method)
{ case "GET": response = request.when().get(endpoint).then().extract().response(); break;
case "POST": response = request.when().post(endpoint).then().extract().response(); break;
case "PUT": response = request.when().put(endpoint).then().extract().response(); break; case
"DELETE": response = request.when().delete(endpoint).then().extract().response(); break; }
attachToAllure(method, endpoint); } private void attachToAllure(String method, String
endpoint) { try { String req = method + " " + baseUrl + endpoint; if (lastBody != null) req
+= "\n\nBody:\n" + lastBody; Allure.addAttachment("Request", "text/plain", req);
Allure.addAttachment("Response [" + response.getStatusCode() + "]", "application/json",
response.getBody().asPrettyString()); } catch (Exception e) { LogUtils.debug("Allure attach
falhou: " + e.getMessage()); } } }
```



## Decisões de design:

- O `baseUri` é setado por instância (não estático), permitindo que diferentes Services apontem para APIs diferentes no mesmo cenário.
- O método `execute()` centraliza toda execução HTTP — logging e Allure acontecem em um único ponto. Isso evita duplicação.
- O `attachToAllure()` envolve tudo em try-catch porque o Allure não deve quebrar o teste se falhar ao anexar.
- A classe é instanciada por cenário via PicoContainer, garantindo isolamento total entre testes.
- O `lastBody` armazena o payload para exibir no attachment — útil para debug de falhas.

**Dica:** Nunca instancie `RestClient` manualmente. Declare-o no construtor do Step e o PicoContainer faz o resto.

## 3.3 Padrão Client-Service

O framework separa responsabilidades HTTP (Client) de lógica de negócio (Service). Essa separação permite que o mesmo Client seja reusado por diferentes Services, e que os Services sejam testados independentemente.

[Diagrama Mermaid — visualize no arquivo .md]

| Camada  | Responsabilidade                  | Sabe sobre HTTP? | Sabe sobre negócio? |
|---------|-----------------------------------|------------------|---------------------|
| Steps   | Orquestrar ações do cenário       | Não              | Indiretamente       |
| Service | Montar requests e definir fluxos  | Parcialmente     | Sim                 |
| Client  | Executar HTTP e capturar resposta | Sim              | Não                 |

A vantagem é clara: se amanhã a API trocar de REST para GraphQL, apenas o Client muda. Se a regra de negócio mudar (ex: campo obrigatório novo), apenas o Service muda. Os Steps permanecem intactos.

**Dica:** Um Service nunca deve fazer asserções. Ele executa a ação e retorna. Quem verifica o resultado é o Step (via `restClient.getStatusCode()`).

## 3.4 PostService

O `PostService` encapsula toda a lógica de negócio relacionada ao recurso `/posts`. Ele sabe QUAIS endpoints chamar, COM QUAIS dados, mas delega a execução HTTP para o `RestClient`.

```
package api.services; import api.clients.RestClient; import utils.JsonUtils; /** * Service para o recurso /posts. * Encapsula lógica de negócio das chamadas API. */ public class PostService { private final RestClient client; public PostService(RestClient client) { this.client = client; } public void listAll() { client.get("/posts"); } public void getById(int id) { client.get("/posts/" + id); } public void getByUser(int userId) { client.get("/posts?userId=" + userId); } public void create() { String body = JsonUtils.load("payloads/posts/create-post.json"); client.setBody(body); client.post("/posts"); } public void update(int id) { String body = JsonUtils.load("payloads/posts/update-post.json").replace("\"id\":1", "\"id\":\"" + id); client.setBody(body); client.put("/posts/" + id); } public void delete(int id) { client.delete("/posts/" + id); } }
```

#### Decisões de design:

- O `Service` recebe o `RestClient` via construtor (injeção de dependência). Isso permite que o `PicoContainer` gerencie o ciclo de vida e garanta que `Steps` e `Service` compartilhem a MESMA instância do `client`.
- Payloads são carregados de arquivos JSON externos via `JsonUtils.load()`. Isso separa dados de lógica e facilita manutenção.
- O `update()` faz replace do ID no JSON — uma estratégia simples para parametrizar payloads sem montar objetos complexos.
- Cada método tem nome semântico: `create()`, `listAll()`, `getByUser()`. Os `Steps` ficam legíveis: `postService.create()`.

**Dica:** Para APIs com muitos endpoints (10+), considere dividir em sub-services: `PostQueryService` (GETs) e `PostCommandService` (POST/PUT/DELETE).

## 3.5 PostRequest — Modelo POJO

O `PostRequest` é um Plain Old Java Object (POJO) que representa a estrutura de dados de um post. Ele serve como modelo tipado quando você precisa construir payloads programaticamente em vez de carregá-los de arquivo.

```
package api.models; /** * Modelo de request para Post. */ public class PostRequest { private String title; private String body; private int userId; public PostRequest() {} public PostRequest(String title, String body, int userId) { this.title = title; this.body = body; this.userId = userId; } public String getTitle() { return title; } public void setTitle(String title) { this.title = title; } public String getBody() { return body; } public void setBody(String body) { this.body = body; } public int getUserId() { return userId; } public void setUserId(int userId) { this.userId = userId; } public String toJson() { return "{\"title\":\"" + title + "\",\"body\":\"" + body + "\",\"userId\":\"" + userId + "\"}"; } }
```

#### Decisões de design:

- O construtor vazio permite deserialização por bibliotecas como Jackson/Gson. O construtor com argumentos permite criação direta.
- O método `toJson()` gera JSON manualmente — simples e sem dependência de biblioteca de serialização. Para POJOs pequenos isso é suficiente.
- Getters e setters seguem a convenção JavaBeans, garantindo compatibilidade com qualquer framework de serialização.
- O campo `body` no POJO tem o mesmo nome do campo no JSON da API. Essa correspondência é intencional para facilitar mapeamento.

**Dica:** Para APIs com payloads grandes (10+ campos), considere usar Jackson com `ObjectMapper` em vez de `toJson()` manual. A chance de erro com concatenação de strings aumenta com a complexidade.

### 3.6 PostBuilder — Builder Pattern com Faker

O `PostBuilder` implementa o Builder Pattern combinado com `JavaFaker` para gerar dados de teste dinâmicos. Ele permite criar posts com dados aleatórios (padrão) ou customizados (quando o cenário exige valores específicos).

```
package api.builders; import api.models.PostRequest; import com.github.javafaker.Faker;
import java.util.Locale; /** * Builder para dados de Post. * Gera dados dinâmicos via Faker
ou permite customização. */ public class PostBuilder { private static final Faker faker = new
Faker(new Locale("pt-BR")); private String title; private String body; private int userId;
private PostBuilder() { this.title = faker.lorem().sentence(5); this.body =
faker.lorem().paragraph(2); this.userId = 1; } public static PostBuilder valid() { return new
PostBuilder(); } public PostBuilder withTitle(String title) { this.title = title; return
this; } public PostBuilder withBody(String body) { this.body = body; return this; } public
PostBuilder withUserId(int userId) { this.userId = userId; return this; } public PostRequest
build() { return new PostRequest(title, body, userId); } }
```

#### Exemplo de uso:

```
// Post com dados totalmente aleatórios PostRequest random = PostBuilder.valid().build(); //
Post com título específico, resto aleatório PostRequest custom = PostBuilder.valid()
.withTitle("Meu Título Fixo").withUserId(5).build();
```

#### Decisões de design:

- O construtor é privado — a única forma de criar um Builder é via `PostBuilder.valid()`. Isso garante que todo post começa com dados válidos.
- O Faker usa `Locale("pt-BR")` para gerar dados em português (nomes, textos).
- O padrão Builder (`withX().withY().build()`) é mais legível que construtores com muitos parâmetros.
- A separação Builder/Model permite ter múltiplos builders para o mesmo modelo: `PostBuilder.valid()`, `PostBuilder.invalid()`, `PostBuilder.minimal()`.

**Dica:** Dados dinâmicos (*Faker*) evitam o problema de "dados viciados" — quando testes passam apenas com dados específicos e falham com dados reais.

### 3.7 JsonUtils

O `JsonUtils` é um utilitário para carregar arquivos JSON do classpath. Ele abstrai a leitura de arquivos e lança exceção semântica quando o arquivo não existe.

```
package útils; import exceptions.FrameworkException; import java.io.InputStream; import
java.util.Scanner; /** * Utilitário para leitura de arquivos JSON do classpath. */ public
class JsonUtils { private JsonUtils() {} /** * Carrega um arquivo JSON do classpath. * @param
path caminho relativo (ex: "payloads/posts/create-post.json") */ public static String
load(String path) { InputStream input =
JsonUtils.class.getClassLoader().getResourceAsStream(path); if (input == null) { throw new
FrameworkException("Arquivo JSON não encontrado: " + path); } try (Scanner scanner = new
Scanner(input, "UTF-8")) { return scanner.useDelimiter("\\A").next(); } } }
```

#### Decisões de design:

- Usa `getClassLoader().getResourceAsStream()` que busca no classpath (dentro de `src/test/resources/`). Isso funciona tanto local quanto em JARs.
- O `Scanner` com delimiter `\\A` (início do input) lê o arquivo inteiro de uma vez — técnica concisa para Java 8.
- Lança `FrameworkException` com mensagem clara em vez de retornar null. Falhar rápido com mensagem explicativa é melhor que `NullPointerException` 5 linhas depois.
- Construtor privado porque a classe é utilitária (apenas métodos estáticos).

**Dica:** Sempre use caminhos relativos ao classpath (ex: `payloads/posts/create-post.json`), nunca caminhos absolutos do sistema de arquivos. Isso garante portabilidade entre máquinas.

### 3.8 Payloads externalizados

Payloads JSON ficam em arquivos separados dentro de `src/test/resources/payloads/`. Isso permite que analistas editem dados sem mexer em código Java, e facilita versionamento.

#### `create-post.json`:

```
{ "title": "Post de Teste Automatizado", "body": "Conteúdo via REST Assured", "userId": 1 }
```

#### `update-post.json`:

```
{ "id": 1, "title": "Título Atualizado", "body": "Corpo atualizado", "userId": 1 }
```

#### Quando usar arquivo JSON vs Builder:

O framework oferece duas estratégias para gerar payloads. A escolha depende do cenário:

- **Arquivo JSON:** Para dados fixos e determinísticos que o cenário valida explicitamente (ex: "o título deve ser X").
- **Builder + Faker:** Para dados dinâmicos onde o valor exato não importa (ex: teste de performance, carga).

O `PostService.create()` usa arquivo JSON porque a feature valida o título retornado. Se usasse Faker, o título seria aleatório e a asserção `"title" deve ter valor "Post de Teste Automatizado"` falharia.

**Dica:** Nomeie payloads com o padrão `{ação}-{recurso}.json`. Isso cria uma convenção previsível: `create-user.json`, `update-order.json`, `partial-update-product.json`.

### 3.9 JSON Schema Validation

A validação de schema garante que a API retorna a estrutura correta independentemente dos VALORES. É um teste de contrato: a API prometeu retornar campos X, Y, Z com tipos específicos.

**post-schema.json:**

```
{ "type": "object", "required": ["userId", "id", "title", "body"], "properties": { "userId": { "type": "integer", "minimum": 1 }, "id": { "type": "integer", "minimum": 1 }, "title": { "type": "string", "minLength": 1 }, "body": { "type": "string", "minLength": 1 } }, "additionalProperties": true }
```

**Step de validação:**

```
@E("a resposta deve estar de acordo com o schema {string}") public void validateSchema(String schemaFile) { restClient.getResponse().then().assertThat().body(io.restassured.module.jsv.JsonSchemaValidator.matchesJsonSchemaInClasspath("schemas/" + schemaFile)); }
```

**O que o schema valida:**

| Regra                          | Significado                            |
|--------------------------------|----------------------------------------|
| <code>"required": [...]</code> | Esses campos DEVEM existir na resposta |
| <code>"type": "integer"</code> | O campo deve ser numérico inteiro      |
| <code>"type": "string"</code>  | O campo deve ser texto                 |
| <code>"minimum": 1</code>      | O valor numérico deve ser $\geq 1$     |
| <code>"minLength": 1</code>    | A string não pode ser vazia            |

"additionalProperties": true

Permite campos extras (não quebra se API adicionar campos)

A validação de schema é complementar à validação de valores. O schema garante ESTRUTURA, os outros steps garantem CONTEÚDO.

**Dica:** Coloque `"additionalProperties": true` nos schemas. Isso evita que o teste quebre quando a API adiciona um campo novo (evolução normal de APIs).

### 3.10 Feature de API em português

A feature de API segue a mesma estrutura BDD da UI, mas sem interação visual. Cada cenário valida um endpoint específico com asserções sobre status code, campos e estrutura.

```
# language: pt @api Funcionalidade: API de Posts Como consumidor da API REST Quero validar os endpoints de posts Para garantir que a API responde corretamente Contexto: Dado que estou consumindo a API de posts @smoke Cenário: Listar todos os posts Quando busco todos os posts Então o status code da resposta deve ser 200 E o Content-Type da resposta deve conter "application/json" E a resposta deve conter 100 posts @smoke Cenário: Buscar um post por ID Quando busco o post de ID 1 Então o status code da resposta deve ser 200 E o campo "userId" deve ter valor inteiro 1 E o campo "id" deve ter valor inteiro 1 E o campo "title" não deve estar vazio E o campo "body" não deve estar vazio Cenário: Buscar posts de um usuário específico Quando busco os posts do usuário 1 Então o status code da resposta deve ser 200 E todos os posts devem ter "userId" igual a 1 Cenário: Criar um novo post Dado que tenho os dados de um novo post Quando envio o novo post Então o status code da resposta deve ser 201 E o campo "title" deve ter valor de texto "Post de Teste Automatizado" E o campo "userId" deve ter valor inteiro 1 E o campo "id" não deve estar vazio Cenário: Atualizar um post existente Dado que tenho os dados de atualização do post 1 Quando atualizo o post 1 Então o status code da resposta deve ser 200 E o campo "title" deve ter valor de texto "Título Atualizado" Cenário: Deletar um post Quando delete o post 1 Então o status code da resposta deve ser 200 Cenário: Buscar post inexistente retorna 404 Quando busco o post de ID 9999 Então o status code da resposta deve ser 404 @smoke Cenário: Validar contrato (schema) do post Quando busco o post de ID 1 Então o status code da resposta deve ser 200 E a resposta deve estar de acordo com o schema "post-schema.json"
```

#### Decisões de design:

- O `contexto` configura a API uma única vez para todos os cenários (define `baseUrl`).
- Cenários cobrem o CRUD completo: Create (POST 201), Read (GET 200), Update (PUT 200), Delete (DELETE 200).
- O cenário 404 valida o caminho negativo — essencial para garantir que a API retorna erro adequado.
- Steps de validação são genéricos ("o campo X deve ter valor Y") — reutilizáveis para qualquer endpoint.

**Dica:** Organize cenários do mais simples ao mais complexo: primeiro GETs (sem body), depois POST/PUT (com body), depois DELETE, depois negativos.